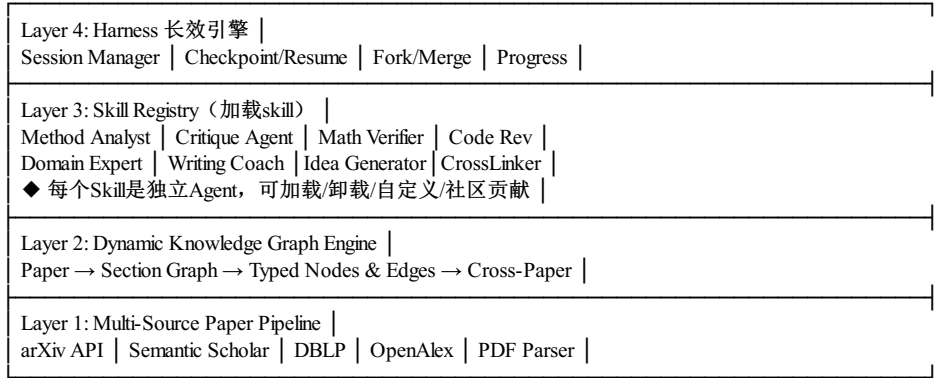


架构：



Layer 1（论文流水线）：负责论文的获取和预处理。支持从arXiv、Semantic Scholar、DBLP、OpenAlex等数据源检索论文，支持PDF文件上传解析。

Layer 2（知识图谱引擎）：负责将论文内容转化为结构化知识表示，提供三种结构化标示：1.将论文内的章节内容图谱化表示。2.跨论文的图谱表示。3.对于用户在merge分支里对一些基础概念，知识点的图谱化表示。

Layer 3（Skill注册中心）：管理所有可用的AI skill。会初始提供一些skill，用户可以根据需要自行修改或插拔。

Layer 4（Harness引擎）：管理长会话生命周期。提供Session创建、Checkpoint保存/恢复、Fork/Merge分支、Progress Log、Initializer Agent等基础设施。

1 Harness长效会话引擎

Session & Checkpoint

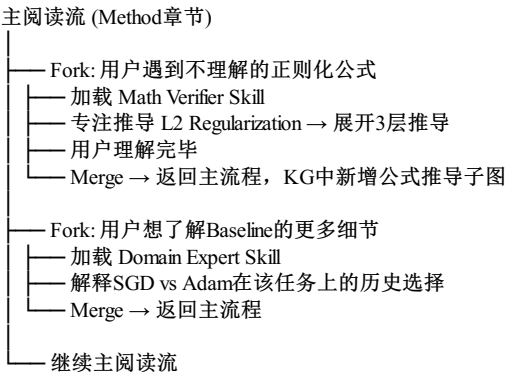
每次阅读结束时，系统自动保存完整的会话状态，包括：

```
Session {
  session_id: uuid,
  paper_id: uuid,
  user_id: uuid,
  state: "active" | "paused" | "completed",
  checkpoints: [
    {
      checkpoint_id: uuid,
      position: { section_id, paragraph_index },
      active_skills: ["method-analyst", "critique-agent"],
      kg_state_snapshot: { expanded_nodes, active_subgraph },
      conversation_history: [...],
      created_at: timestamp
    }
  ],
  progress: { total_sections: 8, completed: 5, percentage: 62.5 }
}
```

用户可以在任何时间暂停阅读，下次恢复时完整还原：读到哪一段了、当时打开了哪些Skill Agent、KG展开到了哪个节点、之前的对话历史。

Fork/Merge分支探索

用户在阅读过程中遇到不理解的公式、概念或需要深入研究的子问题时，可以Fork出一个子会话，加载专门的Skill Agent进行深入探索，完成后Merge回主阅读流。



2 Agent Skill系统

每个Skill是一个独立的Agent角色，带专属的指令体系（System Prompt）、工具链（Tools）和评估标准（Output Schema）。用户可以在阅读过程中动态加载/卸载Skill。

Skill 1: Method Analyst（方法论分析师）

属性	内容
触发时机	自动，进入Method章节时激活
核心行为	拆解Problem Formulation → 画出Method Pipeline → 标注每个Step的Motivation → 输出结构化方法图
输出格式	Markdown结构化分析 + 方法依赖图（DAG）
教学价值	学会「怎么看懂方法论」
交互模式	苏格拉底式反问：不是直接告诉你答案，而是引导你思考「这个Step为什么必要？」

Skill 2: Critique Agent（批判性审稿人）TBD

属性	内容
触发时机	用户主动加载，通常在读完Experiment后
核心行为	以Peer Review标准逐条审查：Assumption是否合理？Baseline是否够强？Ablation是否完整？Claim是否有Evidence支撑？
输出格式	结构化Critique Report + 评分（Novelty/Soundness/Significance/Clarity各1-5分）
教学价值	学会「怎么批判性读论文」——这是本科生最缺的能力
核心创新	不仅给出Critique，还会解释「为什么审稿人会关注这个点」——教用户理解审稿思维

Skill 3: Math Verifier（公式推导验证者）

属性	内容
触发时机	Fork触发，用户遇到公式时
核心行为	Step-by-step展开推导，标注每一步来源和直觉解释，提供简化版数值示例，自动检测推导跳跃
输出格式	三层结构：Layer 1直觉理解 → Layer 2逐步推导（可折叠） → Layer 3数值实例
教学价值	学会「怎么啃公式」——将数学恐惧转化为数学信心
关键能力	自动检测「读者可能不懂的跳跃」，自动插入中间步骤，这是本科生读公式时最大的痛点

Skill 4: Code Reviewer（代码复现审查）

属性	内容
触发时机	用户主动加载，读Implementation章节时
核心行为	检查是否有开源代码 → 如果有，对比代码结构与论文描述的一致性 → 如果没有，基于伪代码评估复现难度和潜在坑点
输出格式	代码一致性报告 + 复现难度评估（Easy/Medium/Hard）+ 潜在坑点清单
教学价值	学会「怎么把论文变成代码」——从论文到实现的关键能力
工具链	GitHub API搜索、代码结构对比、关键超参数提取

Skill 5: Domain Expert（领域知识注入）

属性	内容
触发时机	可配置加载，贯穿全文
核心行为	将论文置于该领域的知识背景中：这个工作承袭了哪条技术路线？与同期工作比独特在哪？属于哪个子方向的哪个阶段？
输出格式	领域上下文标注 + 技术路线图上的位置定位
教学价值	学会「怎么定位论文在领域中的位置」——本科生最常见的困惑是不知道一篇论文在领域中的坐标
关键设计	用户可自定义注入领域知识（如上传导师的研究方向文档作为额外知识库）

Skill 6: Writing Coach（写作教练）TBD

属性	内容
触发时机	用户主动加载，读完整篇后
核心行为	分析论文结构、论证逻辑、图表设计 → 提炼可复用的Writing Pattern → 生成写作模板卡片
输出格式	写作Pattern卡片 + 可复用句式模板 + 图表设计点评
教学价值	学会「怎么写论文」——通过逆向工程顶级论文学习写作
特色功能	自动提取Introduction的「漏斗结构」、Related Work的「分类组织法」、Experiment的「论证链设计」

Skill 7: Idea Generator（创新点生成器）TBD

属性	内容
触发时机	读完后触发
核心行为	基于论文的Limitations + 用户的阅读历史，生成3-5个可做的Follow-up Idea
输出格式	Idea卡片：标题 + 动机（锚定论文原文Limitation）+ 难度（1-5）+ 所需资源 + 潜在坑点 + 预期贡献级别
教学价值	学会「怎么找研究方向」——本科生最常见的困惑之二是不知道读完论文后能做什么
防幻觉设计	每个Idea必须锚定在论文原文的具体Limitation段落，不可凭空生成

Skill 8: Cross-Paper Linker（跨论文连接器）TBD

属性	内容
触发时机	跨论文自动触发
核心行为	检测多篇论文之间的概念共用、方法继承、实验对比、结论冲突 → 自动构建Research Landscape
输出格式	Research Landscape图 + 关联标注（继承/冲突/互补/无关）
教学价值	学会「怎么建立领域认知地图」——从孤立论文到领域全景

核心创新 自动检测跨论文的矛盾结论，高亮提示用户"这两篇论文对同一个问题得出了相反的结论，值得深挖"

用户自定义Skill

- 修改现有Skill：调整System Prompt、增删Output字段
- 创建新Skill：定义全新的Agent角色和指令体系

3 动态知识图谱（Dynamic Knowledge Graph）

节点类型定义（13种Typed Node）

节点类型	含义	关键属性	示例
Problem	研究问题	name, formulation, constraints	"Few-shot Learning: Given K examples per class..."
Method	提出的方法	name, category, pipeline(JSON)	"Prototypical Networks", category: "metric-learning"
Module	方法子模块	name, parent_method, is_contribution	"Feature Extractor (CNN)", is_contribution: false
Baseline	对比基线	name, paper_ref, category	"MAML", category: "classic"
Metric	评估指标	name, domain, higher_is_better	"Accuracy@1", domain: "classification"
Dataset	数据集	name, size, domain, standard_split	"miniImageNet", size: {train:38400,...}
Experiment	实验	experiment_id, setting, results(JSON)	"5-way 1-shot", results: {...}
Figure	图表	figure_id, caption, type, key_insight	"Architecture overview", type: "architecture"
Concept	概念/术语	name, definition, domain	"Attention", domain: "NLP"
Limitation	局限性	description, severity, source	"Only tested on English datasets"
Claim	论文声明	statement, evidence_level	"Our method achieves SOTA", evidence: "experimental"
RelatedWork	相关工作	paper_title, arxiv_id, relationship	"MAML (Finn et al. 2017)", relationship: "precursor"
Insight	用户洞察	content, author(user agent), tags	用户自己记录的理解和想法

5.3.2 边类型定义（9种Typed Edge）

边类型	语义	示例
motivates	A驱动了B的提出	[Problem: Few-shot] → motivates → [Method: ProtoNet]
extends	A是B的技术扩展	[Baseline: MAML] → extends → [Method: ProtoMAML]
outperforms	A在实验中超越B	[Method: ProtoNet] → outperforms → [Baseline: MAML], margin: "+3.2%"
depends_on	A依赖B	[Module: Encoder] → depends_on → [Concept: Self-Attention]
contradicts	A与B存在矛盾	[Claim A] → contradicts → [Claim B], resolution: "..."
ablates	消融实验关系	[Module: Dropout] → ablates → [Experiment: Ablation], effect: "-2.1%"
inspired_by	A给了B灵感	[RelatedWork: ResNet] → inspired_by → [Module: Skip Connection]
evaluated_on	A在B上评测	[Experiment: Table1] → evaluated_on → [Dataset: miniImageNet]
contributes_to	模块对方法的贡献	[Module: Attention] → contributes_to → [Method: ProtoNet], type: "core_innovation"

5.3.3 构建过程(实践中将渐进生成改为一次生成，渐进展开)

用户阅读进度 KG构建动作

Abstract 读完	创建 Problem + Method(壳) 节点
Introduction 读完	创建 Baseline节点集 + RelatedWork节点集，连接 inspires/extends 边
Method §3.1 读完	创建 Module(Encoder)节点 + depends_on Concept 边
Method §3.2 读完	创建 Module(核心创新)节点，连接 contributes_to 边（标记为 core_innovation）
Method §3.3 读完	创建 Module(辅助模块)节点，连接所有 Module→Method 的 contributes_to 边
Experiment 读完	创建 Experiment + Dataset + Metric 节点集，连接 outperforms/ablates/evaluated_on 边
Conclusion 读完	创建 Limitation + Claim 节点集，连接 contradicts 边
多篇阅读后	Cross-Paper Fusion: 合并同名Concept/Dataset/Baseline，检测矛盾和演化链

5.3.4 跨论文KG融合

当用户读了论文A和论文B后，系统自动触发融合：

- 匹配同名Dataset → 合并节点，标注两篇论文在该数据集上的实验设置差异
- 匹配同名Baseline → 创建关联，用户可以对两篇论文对同一Baseline的实验结果
- 检测概念演化链 → 如ProtoNet→ProtoMAML，自动创建extends边并标注key_difference
- 检测矛盾 → 如两篇论文对同一问题得出相反结论，自动创建contradicts边并高亮提示

5.3.5 KG驱动的智能问答

当用户提问时，系统不是在向量库中检索相似文本，而是在知识图谱中进行结构化遍历，找到精确的因果链、实验证据和推理路径。

用户问: "ProtoNet为什么比MAML好?"

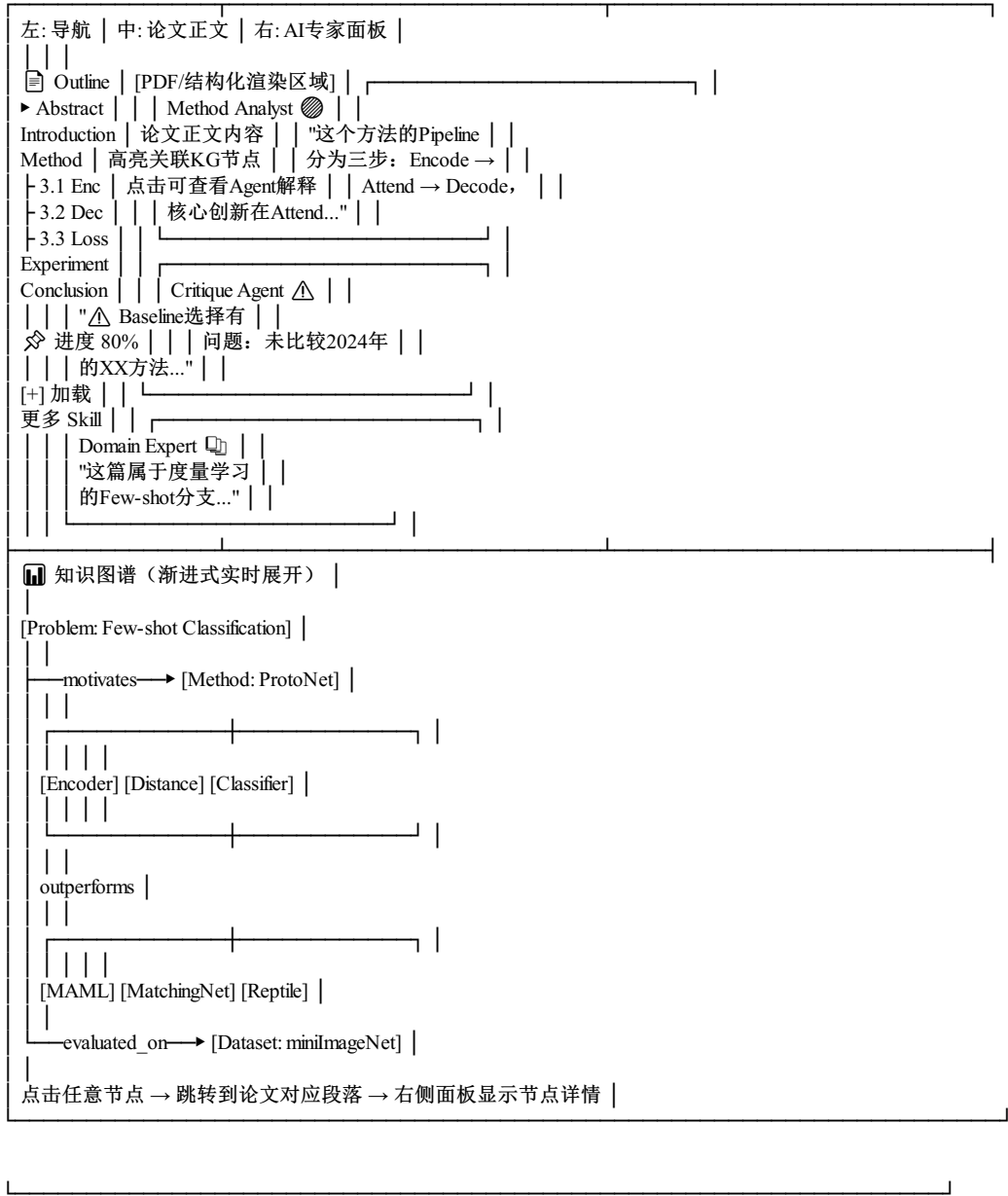
系统执行:

1. KG查询: MATCH (m:Method {name:"ProtoNet"})-[r:outperforms]->(b:Baseline {name:"MAML"})
2. 获取: r.experiment_ref → 定位到 Experiment 节点
3. 获取: r.metric, r.margin → "+3.2% @ Accuracy@1"
4. 遍历: Experiment → evaluated_on → Dataset (查看实验设置)
5. 遍历: Method → contributes_to → Module (查看哪些模块贡献了性能提升)
6. 遍历: Method → depends_on → Concept (查看核心技术差异)

系统回答:
"ProtoNet在miniImageNet 5-way 1-shot上比MAML高3.2% (Acc@1)。
核心差异在于:
1. ProtoNet使用Metric-based方法 (Module: Euclidean Distance),
而MAML使用Optimization-based方法, 在few-shot场景下收敛更慢
2. Ablation实验 (Table 3) 显示, ProtoNet的Feature Extractor结构
贡献了+1.8%, 其余来自度量空间设计
→ [点击查看实验详情] [点击查看MAML原文对此的解释]"

4. 三栏式渐变阅读界面

主界面采用三栏布局, 左侧为阅读导航, 中间为论文正文渲染区, 右侧为AI专家面板:



Fork弹窗模式

用户在主阅读流中遇到需要深入的点时, Fork一个子会话弹窗, 完成探索后无缝返回:

主阅读流 (半透明背景)

